

Practica Patrón State

Víctor Manuel Peiró Martínez

FIIS 2ºA

Diseño De Software

Solución:

Introducción:

En esta entrega, estamos modelando un sistema de petición de libros de una biblioteca usando varios patrones de diseño siendo estos los patrones **Singleton**, **State**, **Observer** y **Strategy**. Cada vez que tengamos un usuario que interactúe con el sistema se le deberá de enviar una notificación en función de la acción que realice. Hay 2 tipos de usuarios que en función de quien sean podrán alquilar el libro un número distinto de días. A continuación explicaremos cómo se han implementado estos patrones.

Patrón Strategy

El patrón Strategy es el patrón encargado del envío de las notificaciones en función del estado de las peticiones. Es decir, en función del estado de la petición se enviará una notificación distinta. El patrón Strategy se divide en 3 componentes que se definen de la siguiente forma:

Estrategias: Es la interfaz/clase abstracta común con los métodos que van a ser definidos en las estrategias concretas. Nuestra estrategia es la clase abstracta **LibraryPushNotificationStrategy** que es la clase en la que luego se definen las estrategias concretas. Las clases **PushNotificationStrategy** y **NotificationStrategy** también se podrían considerar estrategias pero no las usamos directamente ya que son más abstractas y se utilizarían en otros casos en que no estuviésemos modelando la biblioteca. En resumen:

- **NotificationStrategy:** Interfaz. Implementación más abstracta
- **PushNotificationStrategy:** Clase. Desarrolla el método de la interfaz
- **LibraryPushNotificationStrategy:** Clase Abstracta. Añade el método abstracto que será desarrollado por las estrategias concretas

Estrategias Concretas: Es la implementación de la estrategia para un caso concreto. Tenemos en este caso 4 estrategias concretas:

- **CreatedLibraryPushNotificationStrategy:** Envía la notificación de cuando se crea una petición
- **FinishedLibraryPushNotificationStrategy:** Envía la notificación de cuando finaliza una petición
- **PickupLibraryPushNotificationStrategy:** Envía la notificación de cuando se recoge un libro
- **RejectedLibraryPushNotificationStrategy:** Envía la notificación de cuando se rechaza una petición

Contexto: Está compuesto por un objeto de tipo Strategy e instanciado con una estrategia concreta. En nuestro caso, es la clase **LibraryLoanNotification** que se encargará de enviar la notificación correspondiente dependiendo de que estrategia concreta se instancia en su atributo del tipo de la clase abstracta.

Patrón Observer

Hemos implementado para que cuando pase una notificación, se notifique al usuario correspondiente. El sistema de notificaciones a parte de usar el patrón strategy también usar un patrón de tipo pull donde solo se le pasa el estado y no el observador. Sin embargo para que el usuario pueda interpretar las notificaciones, aplicamos el patrón observer en formato push pull donde se pasará el observable y el estado. Para ello tenemos que definir un observer y un observable.

Observer:

El observer es la clase a la que se le notificará de los cambios en el observable. En este caso, el observer se define mediante la interfaz de **PullPushObserver** y nuestro observador va a ser la clase **LibraryUser** que mediante su método `update()` será capaz de imprimir el estado que será la notificación.

Observable

El observable es la clase que contiene la variable de la que se habrá que notificar si ocurre un cambio. En este caso, se define un objeto observable mediante la interfaz **PullPushObservable** que contiene los tres métodos:

- **attach:** Para añadir observers a la lista de observadores
- **detach:** Para eliminar observadores de la lista de observadores
- **notifyObservers:** Avisa a todos los observadores del cambio en la variable de estado.

Nuestra clase observable será el objeto contexto del patrón strategy, es decir la clase

LibraryLoanNotification. Al ser observable, esta clase tiene como atributo una lista de observadores a los que tendrá que avisar cuando acabe su variable estado que es su atributo

LibraryPushNotificationStrategy. Esto se hace en el setter de dicha variable donde se llamará a la función de `notifyObservers()`.

Patrón State

El Patrón State es un patrón de diseño de comportamiento que permite a un objeto cambiar su comportamiento cuando cambia su estado interno. Usamos el patrón state para definir el estado de una petición. Al igual que el patrón Strategy este patrón está compuesto de 3 sectores:

Interfaces de Estado: Son los interfaces que definen los métodos que han de tener los estados. En este caso son 2 interfaces:

- **LibraryLoanState:** Contiene el método para procesar el estado
- **LibraryLoanStateTransition:** Define los métodos que nos pasan de un estado a otro

Estados Concretos: Son las clases que implementan la interfaz de estado y definen el comportamiento específico de cada estado. Como tenemos 5 estados, tenemos una clase por estado que son las siguientes:

- **LibraryLoanCreatedState:** Se ha creado la petición
- **LibraryLoanAdmittedState:** Se ha admitido la petición
- **LibraryLoanProcessedState:** Se ha procesado la petición
- **LibraryLoanRejectedState:** Se ha rechazado la petición
- **LibraryLoanFinishedState:** Se ha finalizado la petición

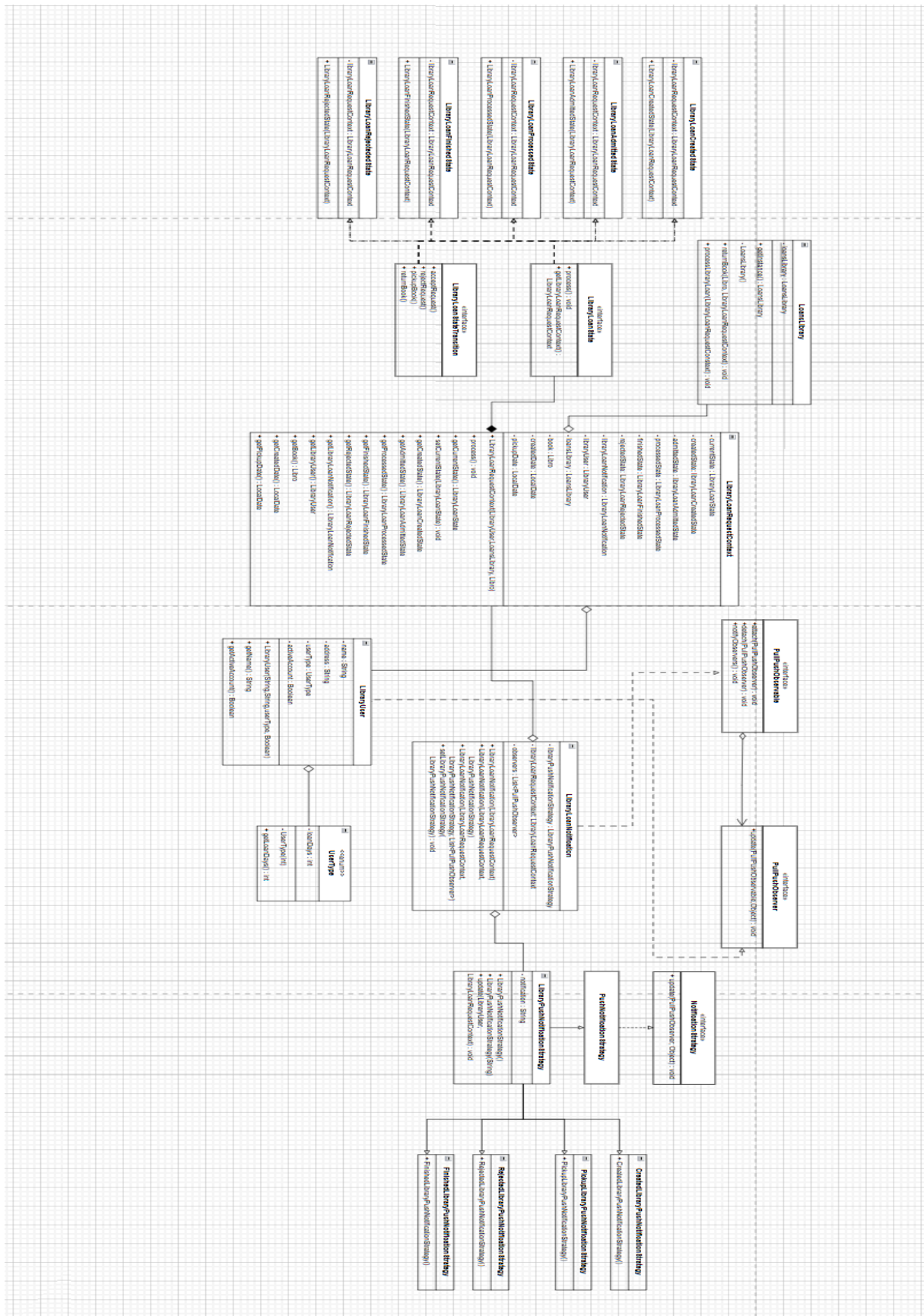
Contexto: Es la clase que mantiene una referencia al estado actual y delega las solicitudes a este estado. En el código, es la clase **LibraryLoanRequestContext**

Patrón Singleton

El patrón Singleton es un patrón de diseño creacional que garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella. Se usa cuando queremos asegurarnos de que solo exista un único objeto de una clase en toda la ejecución del programa.

En nuestro código, podemos ver el patrón singleton en la clase **LoansLibrary** ya que tenemos un constructor privado y el método de getInstance

Para mejor visualización ver archivo adjunto **UML_EntregaState_VMPM.drawio**



Código:

Clase LoansLibrary (Patron Singleton)

```
package EntregaStatePattern;

import EntregaObserverPattern.Libro;
import EntregaObserverPattern.Library;

//Implementa el patron singleton
public class LoansLibrary extends Library {

    private static LoansLibrary loansLibrary = new LoansLibrary(); //Asegura que solo se cree
una instancia del objeto

    //Devuelve la instancia unica
    public static LoansLibrary getInstance() {
        return LoansLibrary.loansLibrary;
    }

    //Constructor privado
    private LoansLibrary() {
        super();
    }

    //Devuelve un libro y procesa la devolucion
    public void returnBook(Libro book, LibraryLoanRequestContext libraryLoanRequestContext)
    {
        super.returnBook(book);
        libraryLoanRequestContext.process();
    }

    //Procesa una prestamp
    public void processLibraryLoan(LibraryLoanRequestContext libraryLoanRequestContext) {
        libraryLoanRequestContext.process();
    }
}
```

Interfaces Patrón Observer

```
package EntregaStatePattern;

//Interfaz plantilla de los observable
public interface PullPushObservable {
    public void attach(PullPushObserver observer);
    public void detach(PullPushObserver observer);
    public void notifyObservers();
}
```

```

package EntregaStatePattern;
//Interfaz plantilla para los observadores
public interface PullPushObserver {
    public void update(PullPushObservable observable, Object state);
}

```

Clase LibraryUser(Observer) y Enumerado de Tipos de Usuario

```

package EntregaStatePattern;
//El usuario es un observer
public class LibraryUser implements PullPushObserver{

    //Atributos
    private String name;
    private String address;
    private UserType userType;
    private Boolean activeAccount;

    //Constructor
    public LibraryUser(String name, String address, UserType userType, Boolean
activeAccount) {
        this.name = name;
        this.address = address;
        this.userType = userType;
        this.activeAccount = activeAccount;
    }

    //Getters y Setters
    public String getName() {
        return this.name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getAddress() {
        return this.address;
    }
    public void setAddress(String address) {
        this.address = address;
    }
    public UserType getUserType() {
        return this.userType;
    }
    public void setUserType(UserType userType) {

```

```

        this.userType = userType;
    }
    public Boolean getActiveAccount() {
        return this.activeAccount;
    }
    public void setActiveAccount(Boolean activeAccount) {
        this.activeAccount = activeAccount;
    }

    //Actualizacion del estado
    public void update(PullPushObservable observable, Object state) {
        System.out.println(state);
    }
}

```

```

package EntregaStatePattern;
//Implementacion del patron singleton para creacion de enumerados con constructor
public enum UserType {
    STUDENT(15),PROFESSOR(10); //Cada objeto del enumerado tiene un valor por defecto de dias

    //Atributo
    private Integer loanDays;

    //Constructor privado del enumerado para poder asignarle el numero de dias a cada tipo de usuario
    private UserType(Integer loanDays) {
        this.loanDays = loanDays;
    }

    //Getter y Setter
    public Integer getLoanDays() {
        return this.loanDays;
    }

    public void setLoanDays(Integer loanDays) {
        this.loanDays = loanDays;
    }
}

```

Patron Strategy - Estrategias

```
package EntregaStatePattern;
//Interfaz para definir notificaciones --> Implementacion mas abstracta
public interface NotificationStrategy {
    public void update(PullPushObserver observer, Object notification);
}

package EntregaStatePattern;
//Clase que desarrolla el metodo definido en la interfaz --> Implementacion menos abstracta
public class PushNotificationStrategy implements NotificationStrategy {
    public void update(PullPushObserver observer, Object notification) {
        observer.update(null, notification);
    }
}

package EntregaStatePattern;
//Clase Abstracta que define las notificaciones --> Estrategia mas concreta
public abstract class LibraryPushNotificationStrategy extends PushNotificationStrategy {

    protected String notification; //Atributo

    //Constructor para notificacion por defecto
    public LibraryPushNotificationStrategy() {
        this("N/A");
    }

    //Constructor
    public LibraryPushNotificationStrategy(String notification) {
        this.notification = notification;
    }

    //Metodo abstracto --> Se define en estrategias concretas
    public abstract void update(LibraryUser libraryUser, LibraryLoanRequestContext
libraryLoanRequestContext);
}
```

Patrón Strategy - Estrategias Concretas

```
package EntregaStatePattern;
//Estrategia Concreta para notificaciones de peticiones creadas
public class CreatedLibraryPushNotificationStrategy extends
LibraryPushNotificationStrategy{

    //Constructor
    public CreatedLibraryPushNotificationStrategy() {
        super();
    }

    //Definicion del metodo update --> Asigna un valor al atributo de la clase abstracta y
    actualiza la estrategia
    public void update(LibraryUser libraryUser, LibraryLoanRequestContext
libraryLoanRequestContext) {
        super.notification = "Notification to user: " + libraryUser.getName() +
            ". Loan admitten on " +
libraryLoanRequestContext.getCreatedDate() +
            "to pickup on: " + libraryLoanRequestContext.getPickupDate();
        super.update(libraryUser, super.notification);
    }
}

package EntregaStatePattern;
import java.time.LocalDate;
//Estrategia Concreta para notificaciones de peticiones finalizadas
public class FinishedLibraryPushNotificationStrategy extends
LibraryPushNotificationStrategy{

    public FinishedLibraryPushNotificationStrategy() {
        super();
    }

    //Definicion del metodo update --> Asigna un valor al atributo de la clase abstracta y
    actualiza la estrategia
    public void update(LibraryUser libraryUser, LibraryLoanRequestContext
libraryLoanRequestContext) {
        super.notification = "Notification to user: " + libraryUser.getName() +
            ". Book: " + libraryLoanRequestContext.getBook() +
            " returned on " + LocalDate.now();
        super.update(libraryUser, super.notification);
    }
}
```

```

package EntregaStatePattern;
import java.time.LocalDate;
//Estrategia Concreta para notificaciones de peticiones de recogida
public class PickupLibraryPushNotificationStrategy extends
LibraryPushNotificationStrategy {

    public PickupLibraryPushNotificationStrategy() {
        super();
    }

    //Definicion del metodo update --> Asigna un valor al atributo de la clase abstracta y
    actualiza la estrategia
    public void update(LibraryUser libraryUser, LibraryLoanRequestContext
libraryLoanRequestContext) {
        super.notification = "Notification to user: " + libraryUser.getName() +
            ". Loan picked up on " + LocalDate.now() +
            " due " +
LocalDate.now().plusDays(libraryUser.getUserType().getLoanDays()) +
            " days (" + libraryUser.getUserType().getLoanDays() + ")";
        super.update(libraryUser, super.notification);
    }
}

package EntregaStatePattern;
import java.time.LocalDate;
//Estrategia Concreta para notificaciones de peticiones rechazadas
public class RejectedLibraryPushNotificationStrategy extends
LibraryPushNotificationStrategy {

    public RejectedLibraryPushNotificationStrategy() {
        super();
    }

    //Definicion del metodo update --> Asigna un valor al atributo de la clase abstracta y
    actualiza la estrategia
    public void update(LibraryUser libraryUser, LibraryLoanRequestContext
libraryLoanRequestContext) {
        super.notification = "Notification to user: " + libraryUser.getName() +
            ". Loan rejected on " + LocalDate.now();
        super.update(libraryUser, super.notification);
    }
}

```

Patrón Strategy - Contexto (Observable)

```
package EntregaStatePattern;

import java.util.List;
import java.util.ArrayList;

//Contexto del Patron Strategy y Observable
public class LibraryLoanNotification implements PullPushObservable{
    //Atributos
    LibraryPushNotificationStrategy libraryPushNotificationStrategy; //Variable Observada
    LibraryLoanRequestContext libraryLoanRequestContext;
    List<PullPushObserver> observers;

    //Constructores
    public LibraryLoanNotification(LibraryLoanRequestContext libraryLoanRequestContext) {
        this(libraryLoanRequestContext, new CreatedLibraryPushNotificationStrategy());
    }

    public LibraryLoanNotification(LibraryLoanRequestContext libraryLoanRequestContext,
    LibraryPushNotificationStrategy pushNotificationStrategy) {
        this(libraryLoanRequestContext, pushNotificationStrategy, new
    ArrayList<PullPushObserver>());
    }

    public LibraryLoanNotification(LibraryLoanRequestContext libraryLoanRequestContext,
    LibraryPushNotificationStrategy pushNotificationStrategy, List<PullPushObserver> observers) {
        this.libraryLoanRequestContext = libraryLoanRequestContext;
        this.libraryPushNotificationStrategy = pushNotificationStrategy;
        this.observers = observers;
    }

    //Añade Observadores a la lista de observadores
    public void attach(PullPushObserver observer) {
        this.observers.add(observer);
    }

    //Elimina un observador de la lista de observadores
    public void detach(PullPushObserver observer) {
        this.observers.remove(observer);
    }

    //Notifica a todos los observadores sobre una cambio en la variable de estado
    public void notifyObservers() {
        for (PullPushObserver observer : observers) {
            this.libraryPushNotificationStrategy.update((LibraryUser)observer,
            this.libraryLoanRequestContext); //Delegacion por agregacion
        }
    }
}
```

```

    }
}

//Getters y Setters
public LibraryPushNotificationStrategy getLibraryPushNotificationStrategy() {
    return this.libraryPushNotificationStrategy;
}

//Cuando hay una cambio en la variable de estado, se notifica a los observadores
public void setLibraryPushNotificationStrategy(LibraryPushNotificationStrategy
libraryPushNotificationStrategy) {
    this.libraryPushNotificationStrategy = libraryPushNotificationStrategy;
    this.notifyObservers();
}

public LibraryLoanRequestContext getLibraryLoanRequestContext() {
    return this.libraryLoanRequestContext;
}

public void setLibraryLoanRequestContext(LibraryLoanRequestContext
libraryLoanRequestContext) {
    this.libraryLoanRequestContext = libraryLoanRequestContext;
}
}

```

Patrón State - Interfaces de los Estados

```

package EntregaStatePattern;
//Define las transiciones entre distintos estados
public interface LibraryLoanStateTransition {
    public void acceptRequest();
    public void rejectRequest();
    public void pickUpBook();
    public void returnBook();
}

package EntregaStatePattern;
//Cada estado ha de ser procesado --> Cada estado ha de poder acceder al contexto para poder
acceder a los estados internos
public interface LibraryLoanState {
    public void process();
    public LibraryLoanRequestContext getLibraryLoanRequestContext();
}

```

Patrón State - Estados Concretos

```
package EntregaStatePattern;
//Estado Rechazado
public class LibraryLoanRejectedState implements
LibraryLoanState,LibraryLoanStateTransition{

    private LibraryLoanRequestContext libraryLoanRequestContext; //Todo estado tiene
que tener acceso al contexto

    //Constructor
    public LibraryLoanRejectedState(LibraryLoanRequestContext context) {
        this.libraryLoanRequestContext = context;
    }

    //No se procesa nada ya que la peticion ha sido rechazada
    @Override
    public void process() {
    }

    //Getter del contexto
    public LibraryLoanRequestContext getLibraryLoanRequestContext() {
        return this.libraryLoanRequestContext;
    }

    //No hay transiciones del estado rechazado
    @Override
    public void acceptRequest() {
        // TODO Auto-generated method stub
    }

    @Override
    public void rejectRequest() {
        // TODO Auto-generated method stub
    }

    @Override
    public void pickUpBook() {
        // TODO Auto-generated method stub
    }

    @Override
    public void returnBook() {
        // TODO Auto-generated method stub
    }
}
```

```

package EntregaStatePattern;
//Estado de Peticion Procesada
public class LibraryLoanProcessedState implements
LibraryLoanState,LibraryLoanStateTransition{

    private LibraryLoanRequestContext libraryLoanRequestContext; //Contexto

    //Constructor
    public LibraryLoanProcessedState(LibraryLoanRequestContext context) {
        this.libraryLoanRequestContext = context;
    }

    //Solo se puede devolver el libro
    @Override
    public void process() {
        this.returnBook();
    }
    public LibraryLoanRequestContext getLibraryLoanRequestContext() {
        return this.libraryLoanRequestContext;
    }
    @Override
    public void acceptRequest() {
        // TODO Auto-generated method stub

    }
    @Override
    public void rejectRequest() {
        // TODO Auto-generated method stub

    }
    @Override
    public void pickUpBook() {
        // TODO Auto-generated method stub

    }

    //Devuelve el libro --> Pasa al estado finalizado y lanza una notificacion
    @Override
    public void returnBook() {

this.libraryLoanRequestContext.getLibraryLoanNotification().setLibraryPushNotificationStra
tegy(new FinishedLibraryPushNotificationStrategy());

```

```

this.libraryLoanRequestContext.setCurrentState(this.libraryLoanRequestContext.getFinished
State());

    }
}

```

```

package EntregaStatePattern;
//Estado Finalizado
public class LibraryLoanFinishedState implements
LibraryLoanState,LibraryLoanStateTransition{

    private LibraryLoanRequestContext libraryLoanRequestContext; //Contexto

    //Constructor
    public LibraryLoanFinishedState(LibraryLoanRequestContext context) {
        this.libraryLoanRequestContext = context;
    }

    //Como la peticion esta finalizada no se procesa nada
    @Override
    public void process() {

    }

    public LibraryLoanRequestContext getLibraryLoanRequestContext() {
        return this.libraryLoanRequestContext;
    }

    @Override
    public void acceptRequest() {
        // TODO Auto-generated method stub
    }

    @Override
    public void rejectRequest() {
        // TODO Auto-generated method stub
    }

    @Override
    public void pickUpBook() {
        // TODO Auto-generated method stub
    }

    @Override
    public void returnBook() {
        // TODO Auto-generated method stub
    }

}

```

```

package EntregaStatePattern;
//Estado De Peticion Creada
public class LibraryLoanCreatedState implements
LibraryLoanState,LibraryLoanStateTransition{

    private LibraryLoanRequestContext libraryLoanRequestContext; //Contexto

    //Constructor
    public LibraryLoanCreatedState(LibraryLoanRequestContext context) {
        this.libraryLoanRequestContext = context;
    }

    //Si el usuario tiene una cuenta activa acepta la peticion, sino la rechaza
    @Override
    public void process() {
        if (libraryLoanRequestContext.getLibraryUser().getActiveAccount()) {
            this.acceptRequest();
        } else {
            this.rejectRequest();
        }
    }

    public LibraryLoanRequestContext getLibraryLoanRequestContext() {
        return this.libraryLoanRequestContext;
    }

    //Pasa al estado admitido y lanza una notificacion
    @Override
    public void acceptRequest() {

this.libraryLoanRequestContext.getLibraryLoanNotification().setLibraryPushNotificationStrategy(new CreatedLibraryPushNotificationStrategy());

this.libraryLoanRequestContext.setCurrentState(this.libraryLoanRequestContext.getAdmittedState());
    }

    //Pasa al estado rechazado y lanza una notificacion
    @Override
    public void rejectRequest() {

this.libraryLoanRequestContext.getLibraryLoanNotification().setLibraryPushNotificationStrategy(new RejectedLibraryPushNotificationStrategy());

```

```

this.libraryLoanRequestContext.setCurrentState(this.libraryLoanRequestContext.getRejected
State());
    }
    @Override
    public void pickUpBook() {
        // TODO Auto-generated method stub

    }
    @Override
    public void returnBook() {
        // TODO Auto-generated method stub

    }
}

```

```

package EntregaStatePattern;

```

```

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

```

```

//Estado admitido

```

```

public class LibraryLoanAdmittedState implements LibraryLoanState,LibraryLoanStateTransition{

```

```

    private LibraryLoanRequestContext libraryLoanRequestContext; //Contexto

```

```

    //Constructor

```

```

    public LibraryLoanAdmittedState(LibraryLoanRequestContext context) {
        this.libraryLoanRequestContext = context;
    }

```

```

    //Si el usuario no pasado con los fias entre la fecha de recogida y la fecha de creacion, se
puede recoger, sino se rechaza

```

```

    @Override

```

```

    public void process() {
        if (ChronoUnit.DAYS.between(LocalDate.now(),
this.libraryLoanRequestContext.getPickupDate()) >= 0) {
            this.pickUpBook();
        } else {
            this.rejectRequest();
        }
    }

```

```

    public LibraryLoanRequestContext getLibraryLoanRequestContext() {
        return this.libraryLoanRequestContext;
    }

```

```

@Override
public void acceptRequest() {
    // TODO Auto-generated method stub

}

//Pasa a estado rechazado y lanza una notificacion
@Override
public void rejectRequest() {

this.libraryLoanRequestContext.getLibraryLoanNotification().setLibraryPushNotificationStrategy(new RejectedLibraryPushNotificationStrategy());

this.libraryLoanRequestContext.setCurrentState(this.libraryLoanRequestContext.getRejectedState());

}

//Pasa a estado procesado y lanza una notificacion
@Override
public void pickUpBook() {

this.libraryLoanRequestContext.getLibraryLoanNotification().setLibraryPushNotificationStrategy(new PickupLibraryPushNotificationStrategy());

this.libraryLoanRequestContext.setCurrentState(this.libraryLoanRequestContext.getProcessedState());
;

}

@Override
public void returnBook() {
    // TODO Auto-generated method stub

}
}

```

Patrón State - Contexto

```

package EntregaStatePattern;

import java.time.LocalDate;
import EntregaObserverPattern.Libro;

//Contexto del patron state
public class LibraryLoanRequestContext {
    private LibraryLoanState currentState; //Estado actual

```

```

//Estados internos
private LibraryLoanCreatedState createdState;
private LibraryLoanAdmittedState admittedState;
private LibraryLoanProcessedState processedState;
private LibraryLoanRejectedState rejectedState;
private LibraryLoanFinishedState finishedState;

//Atributos
private LibraryLoanNotification libraryLoanNotification;
private LibraryUser libraryUser;
private LoansLibrary loansLibrary;
private Libro book;
private LocalDate createdDate;
private LocalDate pickupDate;

//Constructor
public LibraryLoanRequestContext(LibraryUser libraryUser,LoansLibrary loansLibrary,
Libro book) {

    this.createdState = new LibraryLoanCreatedState(this);
    this.admittedState = new LibraryLoanAdmittedState(this);
    this.processedState = new LibraryLoanProcessedState(this);
    this.rejectedState = new LibraryLoanRejectedState(this);
    this.finishedState = new LibraryLoanFinishedState(this);
    this.currentState = this.createdState;

    this.libraryLoanNotification = new LibraryLoanNotification(this);
    this.libraryUser = libraryUser;
    this.loansLibrary = loansLibrary;
    this.book = book;
    this.libraryLoanNotification.attach(libraryUser); //Mete al usuario como observador
    this.createdDate = LocalDate.now();
    this.pickupDate = LocalDate.now().plusDays(2);

}

//Getters y Setters
public LibraryLoanState getCurrentState() {
    return this.currentState;
}

public void setCurrentState(LibraryLoanState currentState) {
    this.currentState = currentState;
}

public LibraryLoanCreatedState getCreatedState() {
    return this.createdState;
}

```

```

    }

    public void setCreatedState(LibraryLoanCreatedState createdState) {
        this.createdState = createdState;
    }

    public LibraryLoanAdmittedState getAdmittedState() {
        return this.admittedState;
    }

    public void setAdmittedState(LibraryLoanAdmittedState admittedState) {
        this.admittedState = admittedState;
    }

    public LibraryLoanProcessedState getProcessedState() {
        return this.processedState;
    }

    public void setProcessedState(LibraryLoanProcessedState processedState) {
        this.processedState = processedState;
    }

    public LibraryLoanRejectedState getRejectedState() {
        return this.rejectedState;
    }

    public void setRejectedState(LibraryLoanRejectedState rejectedState) {
        this.rejectedState = rejectedState;
    }

    public LibraryLoanFinishedState getFinishedState() {
        return this.finishedState;
    }

    public void setFinishedState(LibraryLoanFinishedState finishedState) {
        this.finishedState = finishedState;
    }

    public LibraryLoanNotification getLibraryLoanNotification() {
        return this.libraryLoanNotification;
    }

    public void setLibraryLoanNotification(LibraryLoanNotification libraryLoanNotification) {
        this.libraryLoanNotification = libraryLoanNotification;
    }

    public LibraryUser getLibraryUser() {
        return this.libraryUser;
    }

```

```

    }

    public void setLibraryUser(LibraryUser libraryUser) {
        this.libraryUser = libraryUser;
    }

    public LoansLibrary getLoansLibrary() {
        return this.loansLibrary;
    }

    public void setLoansLibrary(LoansLibrary loansLibrary) {
        this.loansLibrary = loansLibrary;
    }

    public Libro getBook() {
        return this.book;
    }

    public void setBook(Libro book) {
        this.book = book;
    }

    public LocalDate getCreatedDate() {
        return this.createdDate;
    }

    public void setCreatedDate(LocalDate createdDate) {
        this.createdDate = createdDate;
    }

    public LocalDate getPickupDate() {
        return this.pickupDate;
    }

    public void setPickupDate(LocalDate pickupDate) {
        this.pickupDate = pickupDate;
    }

    //Procesa el estado actual
    public void process() {
        this.currentState.process(); //Delegacion por composicion
    }
}

```

Test

```
package EntregaStatePattern;

import EntregaObserverPattern.Libro;
import EntregaObserverPattern.BookState;

public class LibraryLoansTest {
    public static void main(String[] args) {
        LoansLibrary library = LoansLibrary.getInstance(); // singleton
        Libro bookGood= new Libro("Gang of four Design patterns", "Gang of four",450,
BookState.GOOD);
        LibraryUser professor = new LibraryUser("MA",
            "miguel.mesas@u-tad.com", UserType.PROFESSOR, true);
        LibraryUser student = new LibraryUser("Inés",
            "ines@live.u-tad.com", UserType.STUDENT, true);
        LibraryLoanRequestContext professorLoanRequest
            = new LibraryLoanRequestContext(professor, library, bookGood );
        LibraryLoanRequestContext studentLoanRequest
            = new LibraryLoanRequestContext(student, library, bookGood);
        //La biblioteca gestiona las reservas recién creadas
        library.processLibraryLoan(studentLoanRequest);
        library.processLibraryLoan(professorLoanRequest);
        //La biblioteca gestiona la recogida del libro del docente.
        library.processLibraryLoan(professorLoanRequest);
        //La biblioteca gestiona la recogida del libro del alumno.
        library.processLibraryLoan(studentLoanRequest);
        //La biblioteca gestiona la devolución del libro del alumno.
        library.returnBook(studentLoanRequest.getBook(), studentLoanRequest);
        //La biblioteca gestiona la devolución del libro del docente.
        library.returnBook(professorLoanRequest.getBook(), professorLoanRequest);
    }
}
```

Resultado

```
<terminated> LibraryLoansTest [Java Application] C:\Users\Victor Peiro\AppData\Local\Temp\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_22.0.2.v20240802-1626\jre\bin\javaw.exe (21 mar 2025, 16:51:58 - 16:52:00) [pid: 7624]
Notification to user: Inés. Loan admitten on 2025-03-21to pickup on: 2025-03-23
Notification to user: MA. Loan admitten on 2025-03-21to pickup on: 2025-03-23
Notification to user: MA. Loan picked up on 2025-03-21 due 2025-03-31 days (10)
Notification to user: Inés. Loan picked up on 2025-03-21 due 2025-04-05 days (15)
Notification to user: Inés. Book: Libro [nombre=Gang of four Design patterns, autor=Gang of four, numPaginas=450, estado=GOOD] returned on 2025-03-21
Notification to user: MA. Book: Libro [nombre=Gang of four Design patterns, autor=Gang of four, numPaginas=450, estado=GOOD] returned on 2025-03-21
```